CogniShield: Edge-Based Adversarial Immunization and Signed Provenance for Digital Images

BITE497J – Project I | B.Tech. Information Technology

23BIT0030 – Rohit Kumar
23BIT0067 – Shivansh Rai
23BIT0241 – Aviral Bhan

Guide: Dr. Angulakshmi M
School of Computer Science Engineering and Information Systems, VIT Vellore | November 2026

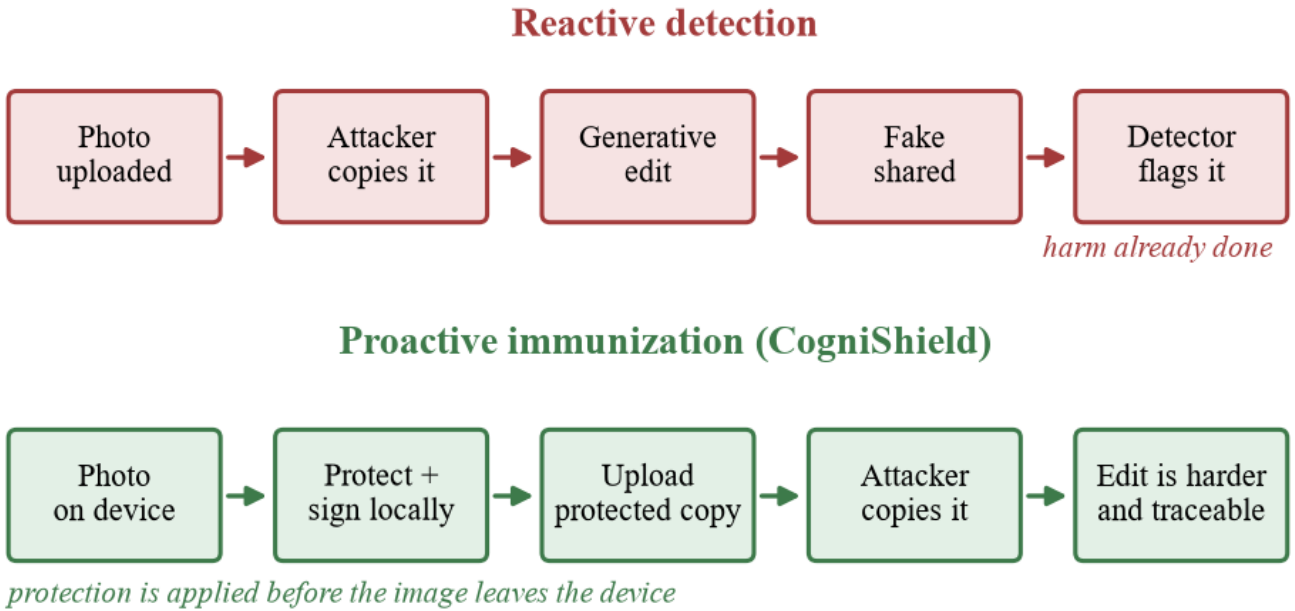
Outline

- The problem and why it matters
- Objective, background, related work
- Scope, threat model and goals
- System design and algorithms
- Provenance and the SD-Turbo test
- Demonstration
- Results on 14 images
- Limitations, future work, conclusion

The problem

- A photo posted online can be copied and edited by anyone
- Image-to-image diffusion models keep the person's face and pose but change the scene
- A few lines of code, or one button, are enough
- Most defences are detectors: they react after the fake already exists
- The owner cannot make the original harder to misuse, or prove which file is theirs

Reactive detection versus acting first



- Detection: arms race with generators, works after harm is done
- CogniShield: change the image slightly before upload and sign it
- Everything runs on the user's own computer

Objective

Build a local tool that protects an image and signs it. It should:

- add a small, bounded, hard-to-see perturbation (at most 8 of 255 levels by default)
- make that perturbation aware of JPEG compression
- measure the quality cost: PSNR, SSIM, MSE, mean and max pixel change
- sign the protected file with Ed25519 over a SHA-256 hash
- optionally compare original and protected images through SD-Turbo
- work from a web page and from the command line, without uploading anything

Motivation

- **Easy tools.** Open-weight models like SD-Turbo run on a home GPU in one to four steps.
- **Detection lags.** Detectors are trained on old generators and weaken after resizing and recompression.
- **Privacy.** Uploading a photo to a protection service sends the very image you worry about to someone else.
- **Learning value.** Adversarial ML, cryptography and web engineering in one small, inspectable pipeline.

Background

Editing with diffusion

- Add noise to the photo, then denoise under a new prompt (SDEdit); "strength" sets how much changes

Adversarial perturbations

- A change of at most ε per pixel can mislead a model; PhotoGuard uses this to resist editing

JPEG compression

- 8×8 DCT blocks; high-frequency detail is quantised away first, exactly where small perturbations live

Provenance

- SHA-256 digest of the file, signed with Ed25519; any changed byte breaks verification

Related work

Work	Idea	Difference from this project
PhotoGuard (2023)	Immunize images against diffusion editing by attacking the model	Needs model access and heavy compute
Fawkes (2020)	Cloak photos against face-recognition training	Different misuse: recognition
Glaze (2023), Anti-DreamBooth (2023), AdvDM (2023)	Protect style and personalised fine-tuning	Target training, not image-to-image editing
Shin & Song (2017)	JPEG-resistant adversarial images	We use the real JPEG encoder in a feedback loop
C2PA	Signed provenance manifests with certificates	We use a local key pair and a sidecar JSON

Problem statement and scope

Given an image, produce a near-identical protected copy that differs by at most a small bound, is built to survive JPEG, has its quality cost measured, and whose exact bytes are signed. All of it runs locally.

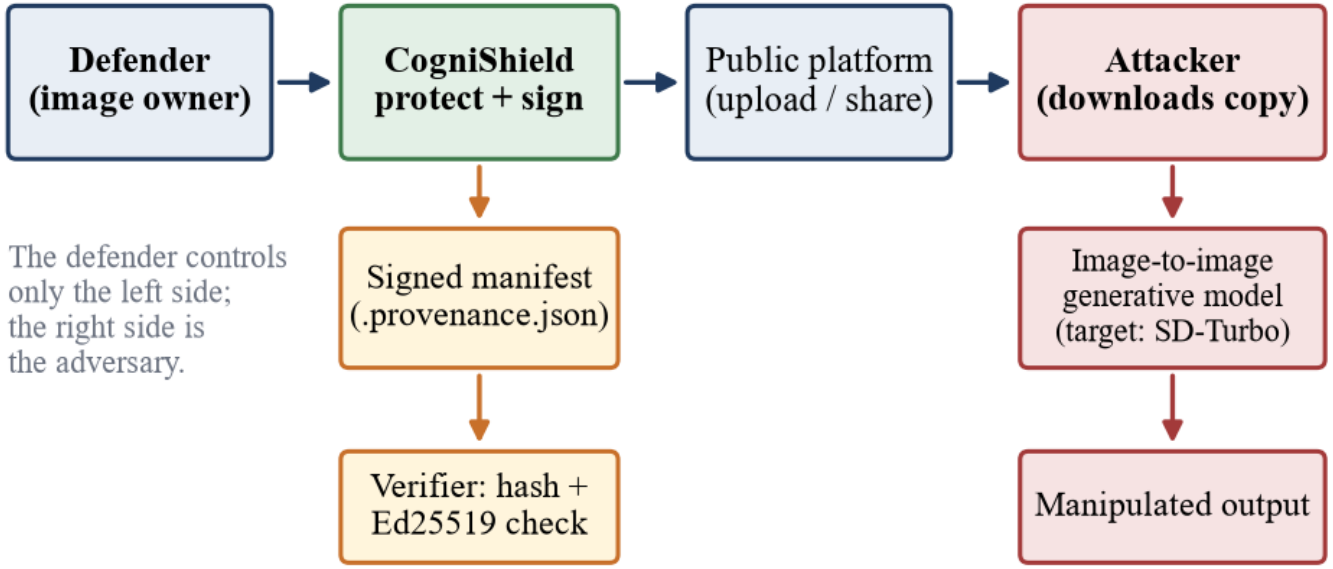
In scope

- Still images (PNG, JPG, WEBP, up to 200 MB)
- Edge-aware perturbation with JPEG feedback
- Metrics and JPEG tests at 95/85/75/60/50
- Ed25519 signing and verification
- Optional SD-Turbo comparison

Out of scope

- Video and audio
- Gradient optimisation against a model
- Guarantees against cropping, screenshots, print-and-scan
- Certificate authority / PKI
- Human perceptual study

Threat model

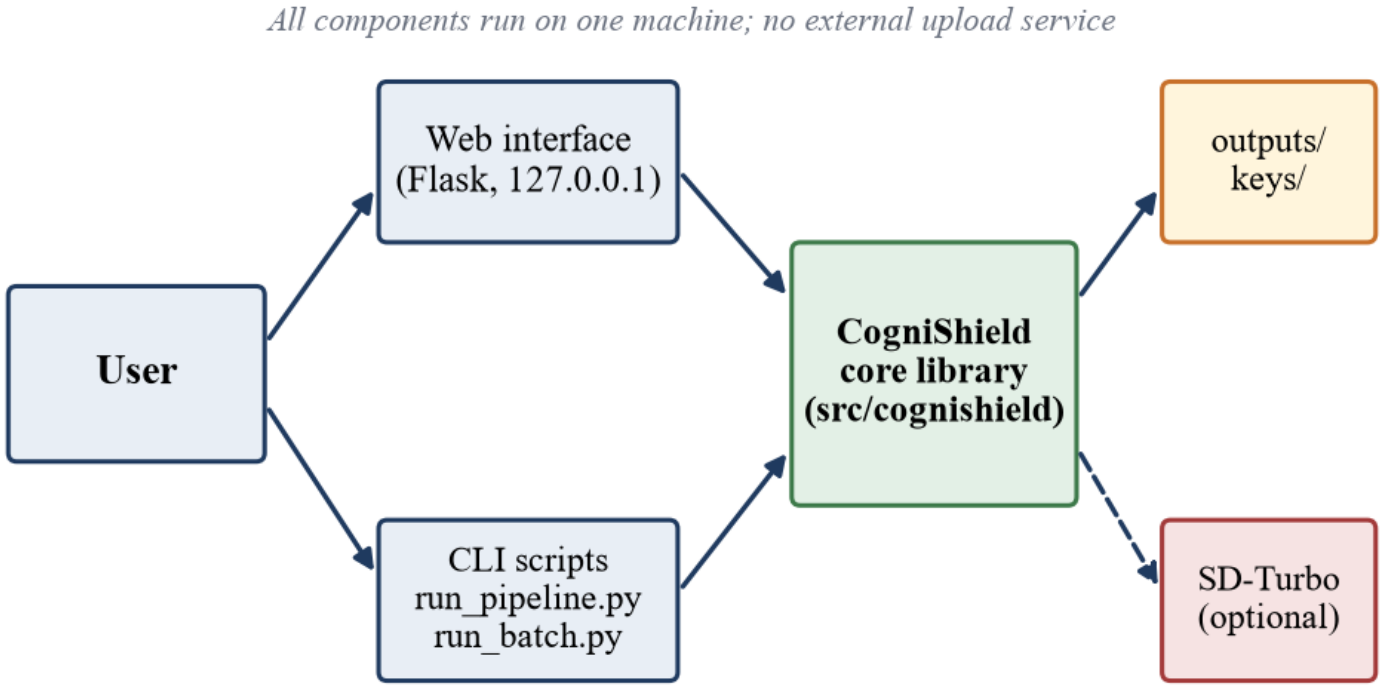


- Attacker downloads the published image and runs image-to-image generation
- Attacker can recompress with JPEG
- Attacker cannot forge a signature without the private key
- We claim integrity of the file and a measured change, not immunity

Goals and how they are checked

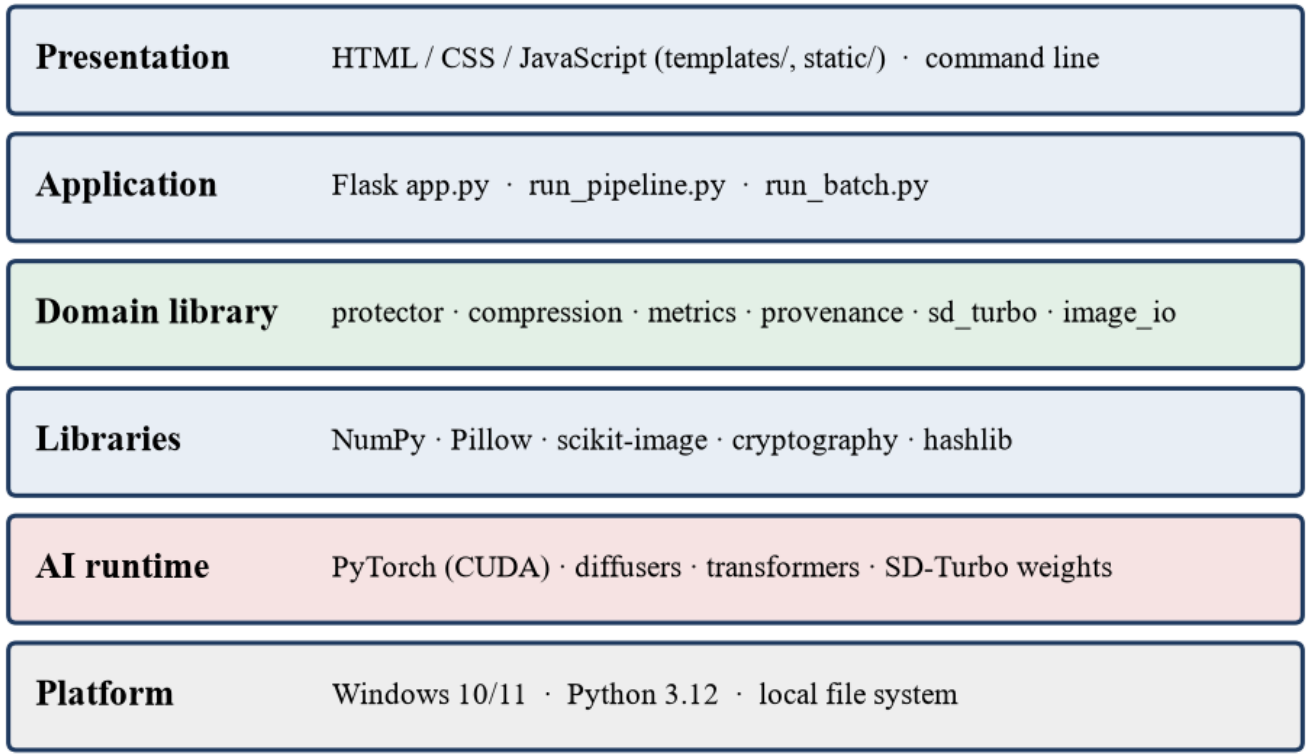
ID	Goal	Acceptance criterion
G1	Imperceptible protection	PSNR above 30 dB, SSIM close to 1.0
G2	Bounded change	Max pixel change never exceeds the budget
G3	Compression awareness	Tested at JPEG 95, 75, 50; effect of feedback measured
G4	Reproducibility	Same image, seed, settings: identical output
G5	Verifiable provenance	Valid for the file, fails if one byte changes
G6	Target-model comparison	Original and protected through SD-Turbo, side by side
G7 / G8	Usability, local operation	Browser workflow; server on 127.0.0.1 only

System overview



- Core library of six small modules
- Flask web page and two command-line programs
- Outputs and keys stay in project folders
- SD-Turbo is optional

Software architecture

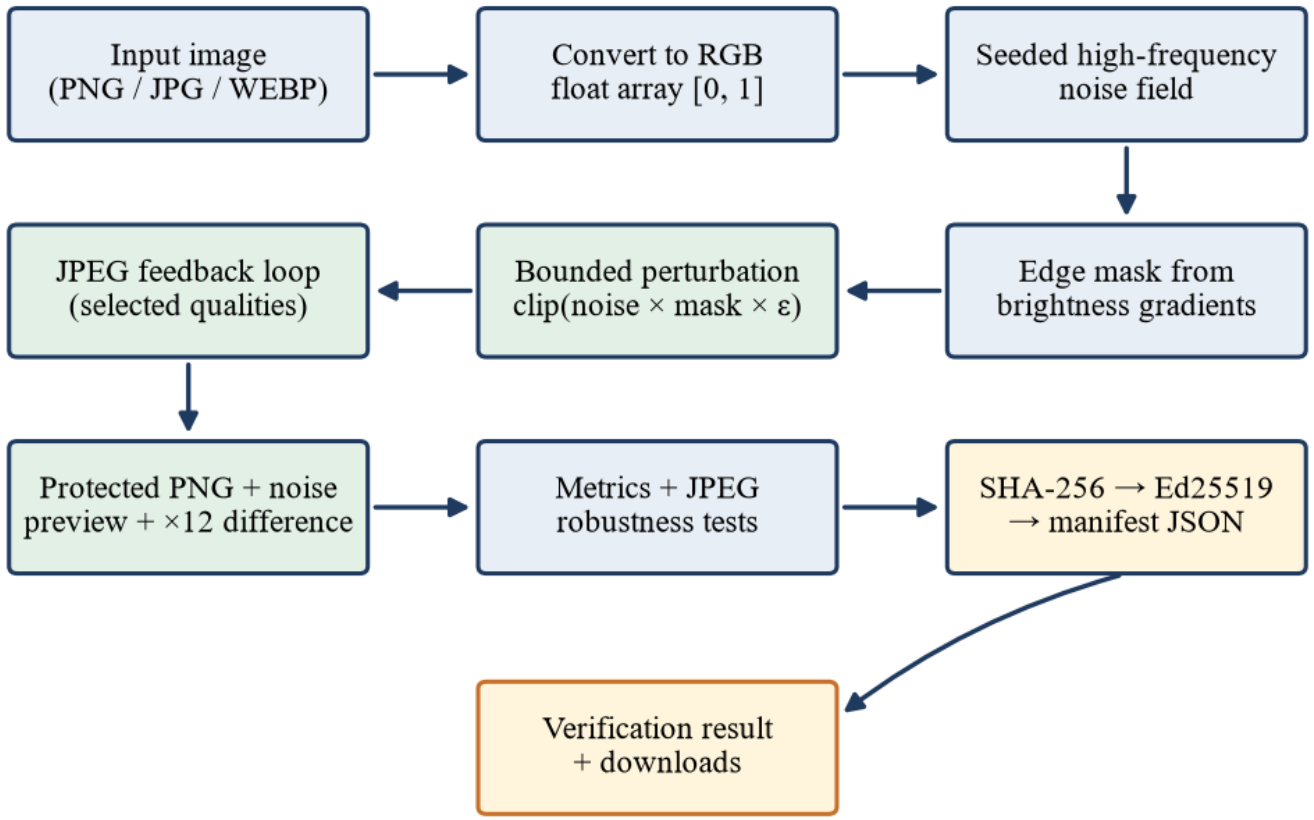


- protector: noise, edge mask, feedback loop
- compression: JPEG round trips
- metrics: PSNR, SSIM, MSE, deltas
- provenance: hash, sign, verify
- sd_turbo: optional model runner
- About 430 lines of Python in the library

Technical specification

	Core protection	SD-Turbo test (optional)
Hardware	Any 64-bit CPU, 4 GB RAM	NVIDIA GPU with CUDA
Software	Windows 10/11, Python 3.12	PyTorch (CUDA 12.8), diffusers
Libraries	NumPy, Pillow, scikit-image, cryptography, Flask	transformers, accelerate, huggingface_hub
Disk	About 2 GB packages, about 5 MB per run	Several GB for the model
Interfaces	Web at 127.0.0.1:5000; run_pipeline.py; run_batch.py	Tick-box in the web page

Processing pipeline

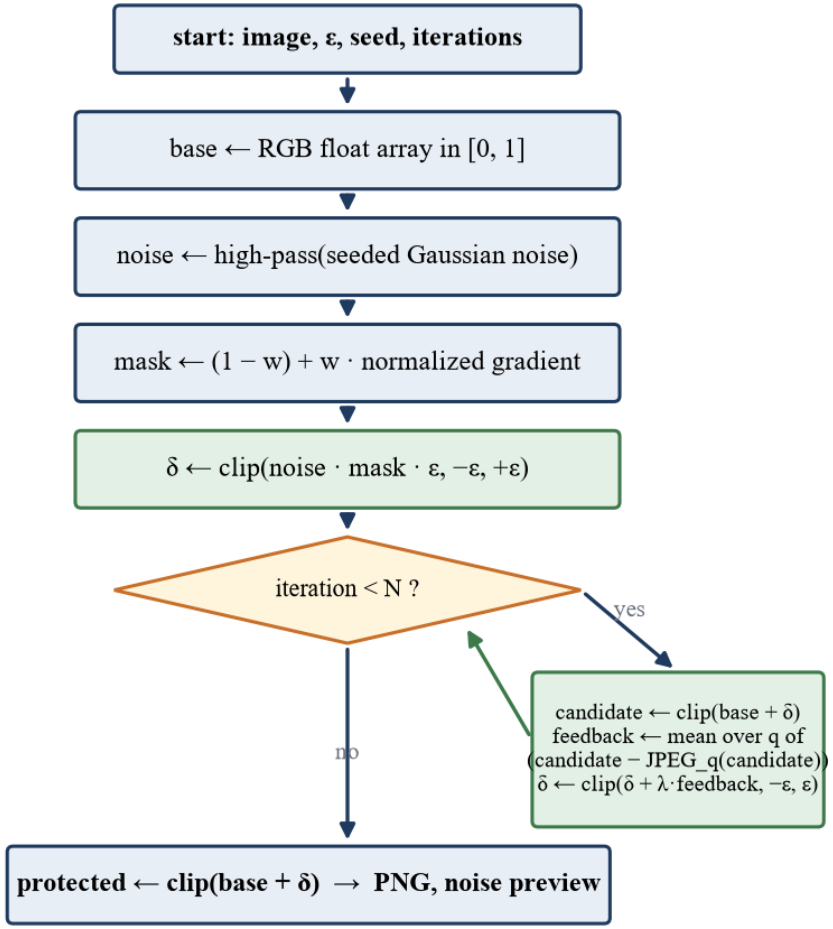


- Image is converted to RGB floats in [0, 1]
- Seeded noise × edge mask × budget
- JPEG feedback refines it
- Protected PNG, noise preview, difference ×12
- Metrics, JPEG tests, then hash and signature

Noise and edge mask

- **High-frequency noise.** Seeded Gaussian noise ξ minus a blurred copy, scaled by its largest value: $n = (\xi - G(\xi)) / \max|\ldots|$
- **Edge mask.** Gradient of the grey image, normalised to g in [0, 1]: $m = (1 - w) + w \cdot g$, with $w = 0.65$, so flat areas still get 35%
- **Bounded start.** $\delta 0 = \text{clip}(n \cdot m \cdot \varepsilon, -\varepsilon, +\varepsilon)$, where $\varepsilon = E/255$ and E is 1 to 16 (default 8)
- Same seed gives the same noise, so every run can be repeated

JPEG feedback loop



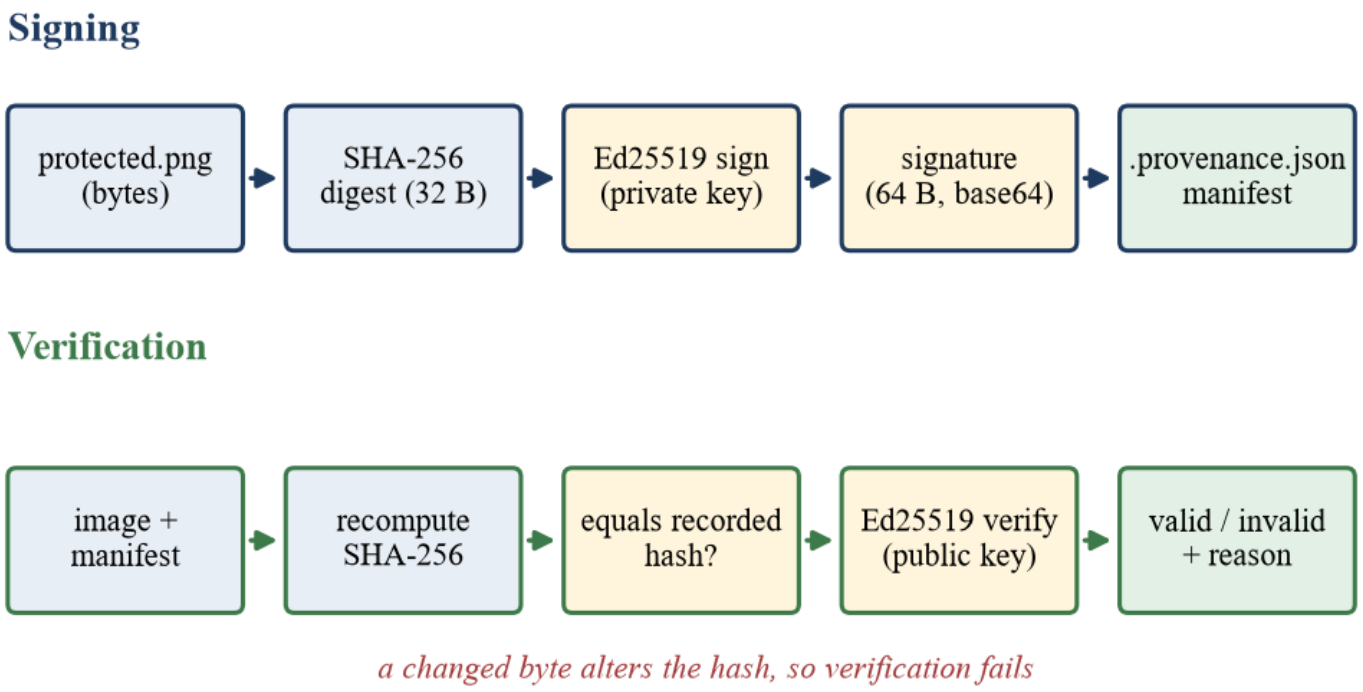
- Candidate = clip(original + δ)
- Compress it at each chosen quality; average what JPEG removed: f = mean(candidate − JPEG_q(candidate))
- Update δ ← clip(δ + λ·f, −ε, +ε), with λ = 0.16
- Repeat N times (default 4); uses the real JPEG encoder, no gradients
- Side effect, measured later: the loop adds energy, so PSNR falls as iterations rise

How quality is measured

Measure	Good direction	Meaning
PSNR	Higher	10·log10(1 / MSE); above 30 dB usually means low visible distortion
SSIM	Closer to 1.0	Local luminance, contrast and structure similarity
MSE	Lower	Mean squared pixel error on the 0–1 scale
Mean abs. delta	Lower	Average per-pixel change
Max abs. delta	Lower	Largest change; equals the budget here

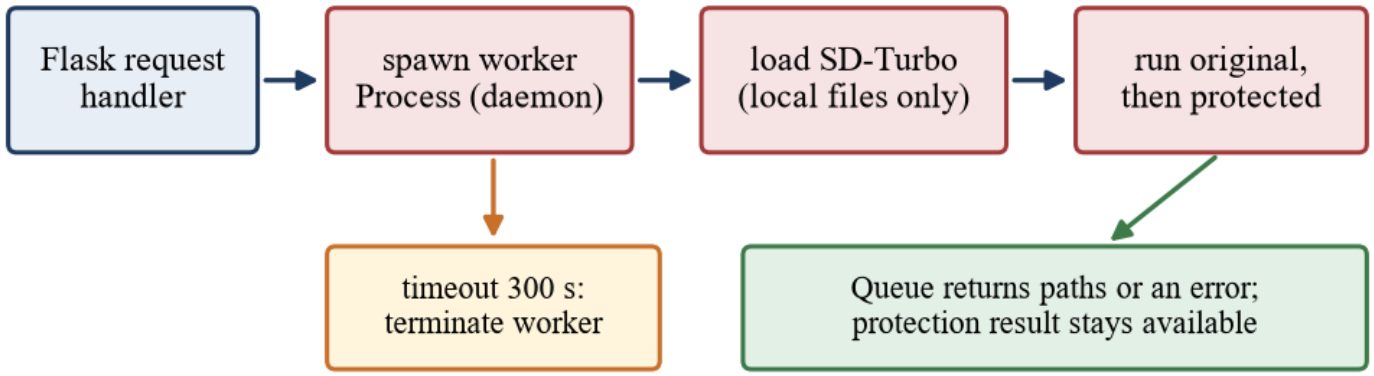
The protected image is also JPEG-compressed at 95, 75 and 50 and compared with the original.

Signed provenance



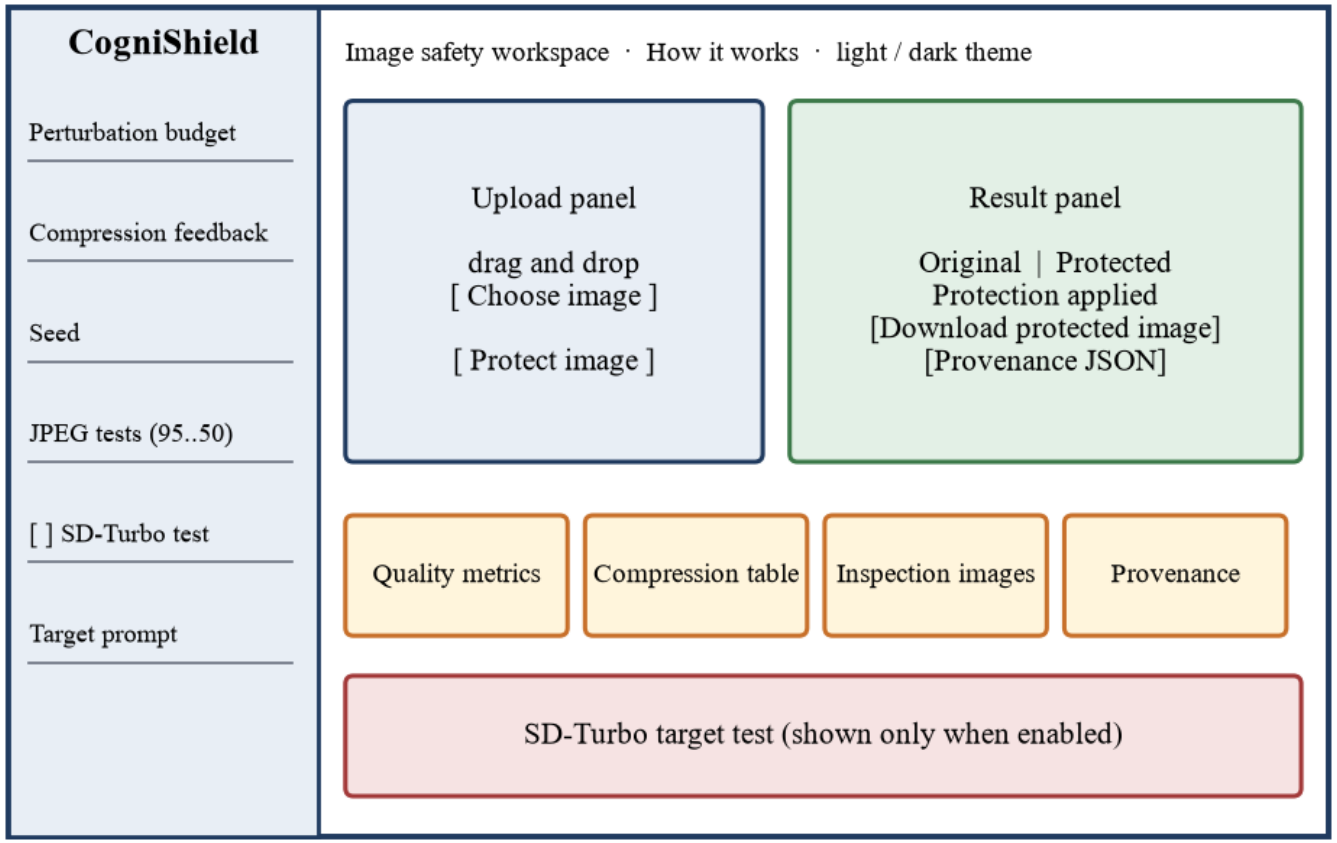
- SHA-256 of the protected PNG bytes
- Ed25519 signature (64 bytes), saved in a JSON manifest
- Verification recomputes the hash first, then checks the signature
- Proves the bytes match what was signed; says nothing about whether the scene is real

Optional SD-Turbo test



- Image-to-image, strength 0.55, guidance 0, longest side 512 px
- Same prompt and seed for original and protected
- Runs in a separate process with a 300 s timeout
- If the model fails, the signed protection result is still returned

Demonstration: web interface



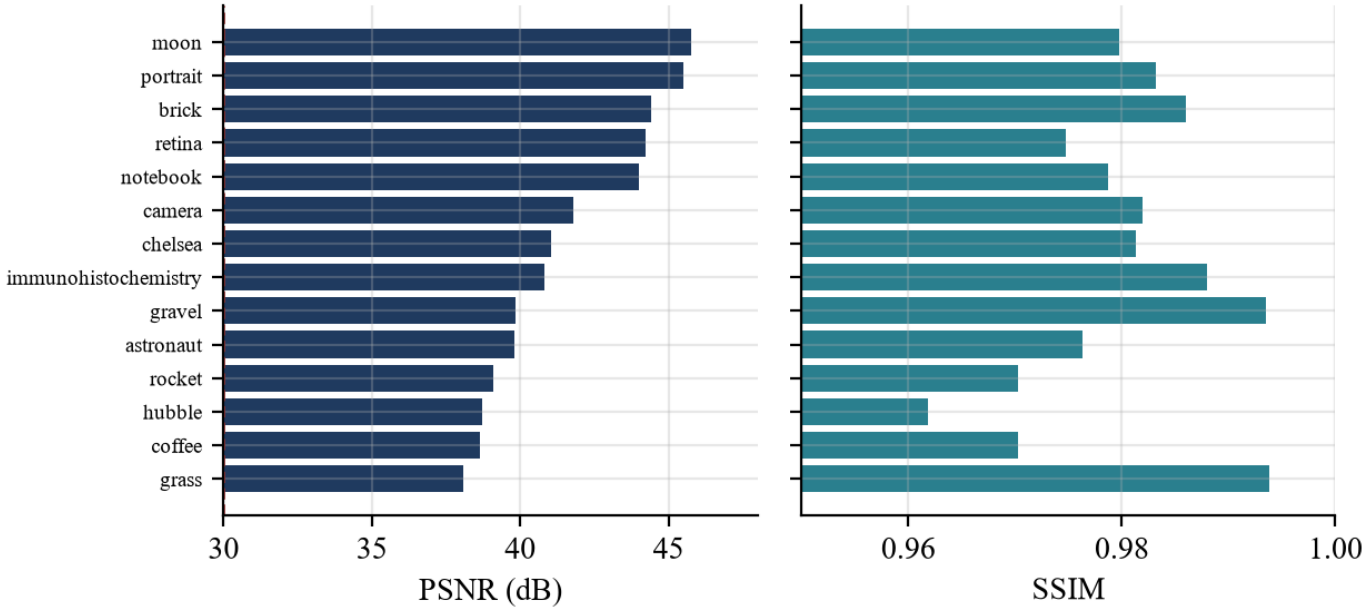
- Drag and drop an image
- Set budget, feedback, seed, JPEG tests
- Press Protect image
- Compare images, read metrics and the JPEG table
- Download the PNG and the manifest
- Dark mode by default, light toggle

Demonstration: portrait run



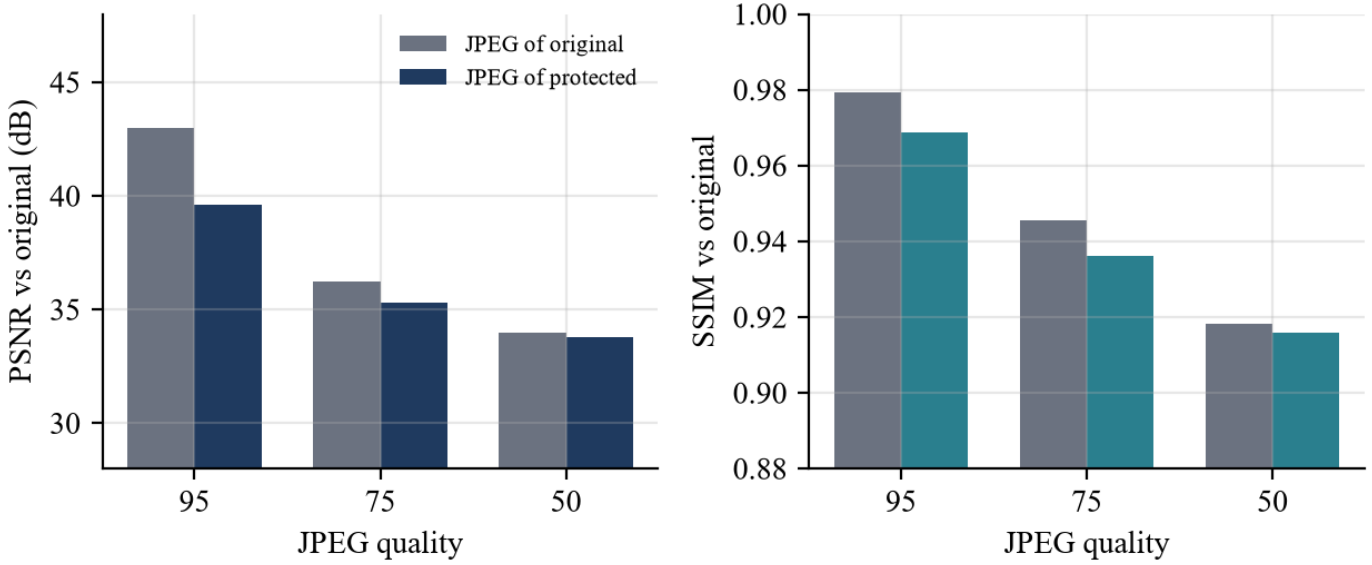
- Default settings: budget 8, 4 iterations, seed 42
- PSNR 45.49 dB, SSIM 0.9831
- Average pixel change about 1.00 level; maximum exactly 8 levels
- Difference ×12 follows hair, eyes and clothing edges
- After JPEG q=95 / 75 / 50: PSNR 45.12 / 42.10 / 39.48 dB

Results: how visible is the change?



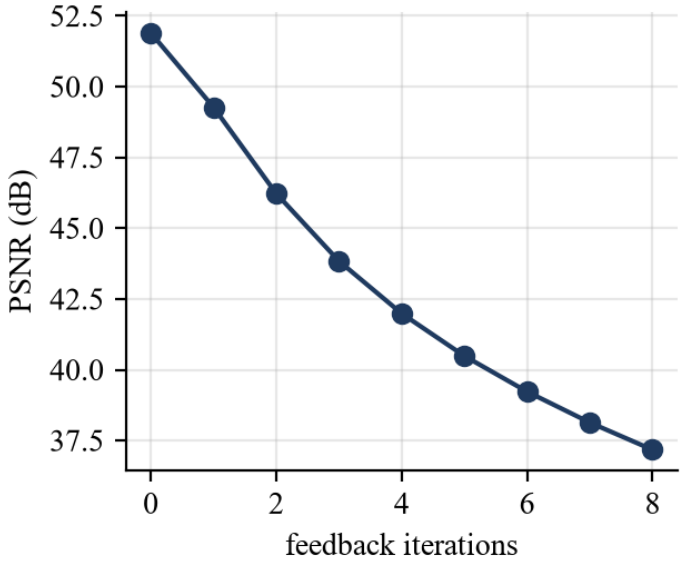
- 14 images, default settings
- Mean PSNR 41.56 dB (range 38.07 to 45.78)
- Mean SSIM 0.9800 (lowest 0.9618)
- All 14 above 30 dB and above 0.95
- Max change exactly 8 levels in every image

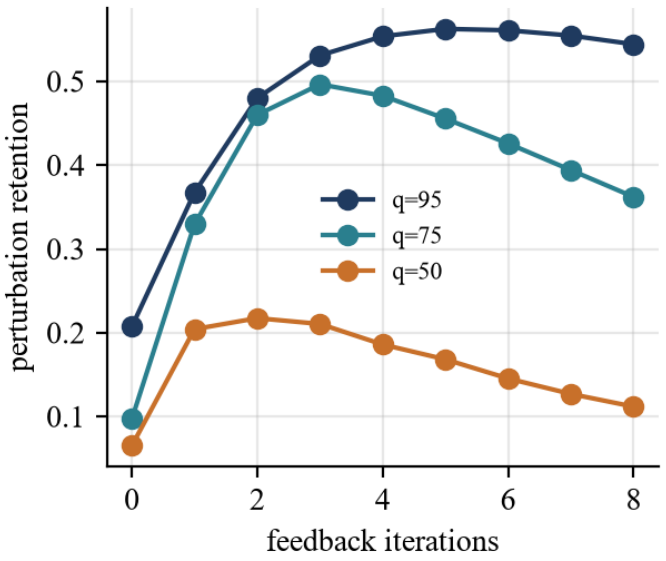
Results: after JPEG compression



- Protected vs original JPEG: PSNR gap 3.37 / 0.94 / 0.16 dB at q = 95 / 75 / 50
- Compression error swamps the perturbation as quality drops
- Median retention of the perturbation: 0.60 / 0.50 / 0.17
- About half survives at 95 and 75; under a fifth at 50

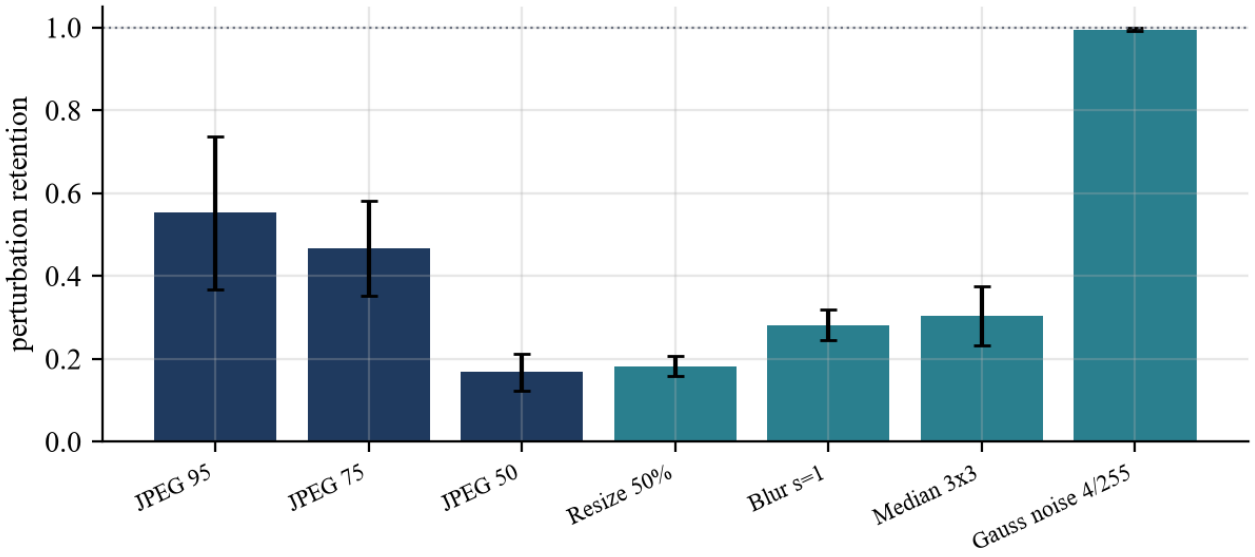
Results: feedback iterations and budget





- Feedback raises retention at q=95 from 0.21 to 0.55, at q=75 from 0.10 to 0.48
- It costs about 9.9 dB of PSNR over 4 iterations
- Benefit peaks at 3 to 5 iterations, then falls
- Budget 1 to 16: PSNR 50.25 to 39.06 dB
- Suggestion: 3 iterations may be a better default

Results: other edits, and the final perturbation



- Resize 50% keeps 0.18, blur 0.28, median filter 0.30 of the perturbation
- Initial noise averages only 0.49 levels; the final perturbation 1.62 levels
- Final perturbation tracks the image's own JPEG residual (correlation 0.75) more than the seeded noise (0.49)
- Only 1.7% of pixel values reach the budget

Results: effect on SD-Turbo

(a) Original through SD-Turbo



(b) Protected through SD-Turbo



- Outputs for original and protected look almost identical
- PSNR between outputs 41.06 dB, SSIM 0.9863
- Between the inputs it was 45.49 dB
- **No visible disruption of SD-Turbo at default settings**
- One image, one prompt, one seed; the harness works, the defence does not yet

Results: provenance, repeatability and speed

Check	Result
Recorded manifests whose SHA-256 matches the file	18 of 18
Digest still matches after one flipped bit	0 of 18
Same image, seed, settings twice	Bit-identical output
PSNR across 10 seeds (portrait)	45.11 to 45.90 dB
Protection time, 512 × 512 / 1024 × 1024	0.19 s / 0.75 s
Average over the 14 test images	0.27 s per image

Limitations and future work

Limitations

- No protective effect on SD-Turbo shown
- Resizing, blur, median filter and JPEG q=50 remove most of it
- Re-encoding invalidates the signature of the copy
- Key is unencrypted; local key proves origin, not identity
- 14 images, one model test, no human study

Future work

- Optimise the perturbation against the target model
- Test face-swap and inpainting models
- Perturbations robust to resizing and filtering
- Add a robust watermark next to the signature
- Open provenance standard, protected keys, on-device version

Conclusion

- A complete local pipeline: perturb, measure, test JPEG, sign, verify
- Meets the goals for imperceptibility, boundedness, reproducibility and integrity
- Honest result: not yet a proven defence against a diffusion editor
- Value: a transparent baseline for stronger, model-aware protection